

Introdução

SQL (Structured Query Language) vs. NoSQL

SQL: O modelo relacional estruturado

O que é o SQL?

 **SQL (Structured Query Language)** é a linguagem utilizada para interagir com bases de dados relacionais.

Estas bases de dados organizam a informação em **tabelas** (semelhantes a folhas de Excel), onde:

- Cada **linha** representa um registo (por exemplo, um cliente)
- Cada **coluna** representa um atributo (por exemplo, nome, email, telefone)
- As tabelas relacionam-se entre si através de **chaves**

Características fundamentais

1 Estrutura rígida (Schema)

- Os dados seguem um esquema predefinido
- Cada coluna tem um tipo de dados específico (texto, número, data, etc.)
- Alterações à estrutura exigem planeamento e podem ser complexas

2 ACID: Garantias de consistência

As bases de dados SQL garantem propriedades conhecidas como ACID:

- **Atomicidade:** Uma transação é executada completamente ou não é executada
- **Consistência:** Os dados respeitam sempre as regras definidas
- **Isolamento:** Transações simultâneas não interferem entre si
- **Durabilidade:** Uma vez confirmados, os dados não se perdem

3 Relacionamentos complexos

- Permitem modelar relações entre entidades (clientes, encomendas, produtos)
- Utilizam chaves primárias e estrangeiras
- Garantem integridade referencial

SQL: O modelo relacional estruturado (cont.)

Vantagens do SQL

- **Consistência garantida**
Ideal para sistemas financeiros, bancários, de faturação
- **Queries complexas**
Permitem interrogações sofisticadas com múltiplas tabelas
- **Transações robustas**
Adequado para operações que exigem precisão
- **Maturidade**
Tecnologia consolidada com vasto suporte e profissionais especializados
- **Normalização**
Evita a redundância de dados

Limitações do SQL

- **Escalabilidade vertical**
Crescer implica servidores mais potentes (caros)
- **Rigidez**
Difícil adaptar-se a dados com estrutura variável
- **Desempenho**
Pode ser mais lento em volumes massivos de leituras
- **Complexidade**
Modelação de dados pode tornar-se complexa em cenários não tabulares

Exemplos de sistemas SQL

PostgreSQL

Open-source, robusto, rico em funcionalidades

Microsoft SQL Server

Ecossistema Microsoft

MySQL/MariaDB

Popular em aplicações web

Oracle Database

Empresarial, elevado desempenho

NoSQL: Flexibilidade e escala

O que é o NoSQL?

i **NoSQL (Not Only SQL)** agrupa sistemas de bases de dados que **não seguem o modelo relacional tradicional**.

Surgiram para responder a desafios modernos:

- Volumes massivos de dados (Big Data)
- Necessidade de escalabilidade horizontal (adicionar mais servidores)
- Dados com estrutura variável ou inexistente
- Requisitos de alta disponibilidade e velocidade

Características fundamentais

1 Schema flexível

- Não exigem estrutura predefinida
- Cada registo pode ter campos diferentes
- Facilita adaptação a mudanças nos requisitos

2 Escalabilidade horizontal

- Distribuem dados por múltiplos servidores
- Permitem crescimento incremental e económico
- Adequado para aplicações web modernas com milhões de utilizadores

3 BASE: Outro modelo de consistência

Ao contrário do ACID, muitos sistemas NoSQL seguem o modelo BASE:

- **Basically Available:** Sistema sempre disponível (mesmo com falhas parciais)
- **Soft state:** Estado pode mudar sem input (reconciliação de dados)
- **Eventually consistent:** Consistência alcançada eventualmente, não imediatamente

Tipos de bases de dados NoSQL

Bases de dados de documentos

❗ Armazenam dados em **documentos** semelhantes a JSON, com pares chave-valor aninhados.

Características

- Estrutura flexível por documento
- Suportam dados complexos e hierárquicos
- Queries por qualquer campo do documento

Quando devemos utilizar?

- Catálogos de produtos (cada produto tem atributos diferentes)
- Gestão de conteúdos (artigos, posts, perfis de utilizador)
- Aplicações móveis com sincronização

Exemplos

- MongoDB (mais popular)
- CouchDB
- Amazon DocumentDB

📄 **Exemplo:** Uma loja online com milhares de produtos diferentes (livros, eletrónica, roupa) onde cada categoria tem atributos específicos. Um livro tem ISBN e autor, um portátil tem processador e RAM. Com documentos, cada produto define os seus próprios campos.

Bases de dados chave-valor

 O modelo mais simples. Cada registo é um **par chave-valor**, como um dicionário gigante.

Características

- Extrema simplicidade
- Desempenho excepcional (leituras/escritas)
- Sem queries complexas (acesso direto por chave)

Quando devemos utilizar?

- Cache de sessões de utilizador
- Configurações de aplicações
- Carrinho de compras temporário
- Contadores e métricas em tempo real

Exemplos

- Redis (também usado como cache)
- Amazon DynamoDB
- Riak

 **Exemplo:** Um site de e-commerce que precisa de guardar temporariamente os produtos no carrinho de cada utilizador (identificado por um ID de sessão). O acesso é sempre direto: "Que produtos tem o utilizador XYZ no carrinho?"

Bases de dados de colunas (Column-family)

 Organizam dados por **colunas** em vez de linhas, agrupando-as em famílias.

Características

- Otimizadas para leitura/escrita de colunas específicas
- Compressão eficiente de dados semelhantes
- Adequadas para análise de grandes volumes

Quando devemos utilizar?

- Sistemas de análise e reporting
- Séries temporais (logs, sensores, métricas)
- Data warehousing
- Recomendações e análise de comportamento

Exemplos

- Apache Cassandra
- HBase
- ScyllaDB

 **Exemplo:** Uma plataforma de streaming que regista todos os eventos de visualização (utilizador, vídeo, timestamp, duração). Precisa de consultar "Quantas visualizações teve cada vídeo por dia?". Acesso rápido a colunas específicas em milhares de milhões de registos.

Bases de dados de grafos

❗ Modelam dados como **nós (entidades)** e **relações (arestas)** entre eles.

Características

- Relações são cidadãos de primeira classe
- Queries sobre conexões (amigos de amigos, caminhos)
- Desempenho em queries de relacionamento

Quando devemos utilizar?

- Redes sociais (conexões entre pessoas)
- Sistemas de recomendação
- Detecção de fraude (padrões de relações suspeitas)
- Gestão de conhecimento e ontologias

Exemplos

- Neo4j
- Amazon Neptune
- ArangoDB

📄 **Exemplo:** Uma rede social profissional (tipo LinkedIn) que precisa de encontrar "Quem são os contactos de 2º grau que trabalham em empresas de tecnologia em Lisboa?". Queries complexas sobre relações entre pessoas, empresas e localizações.

SQL vs. NoSQL: Quando é que devo escolher cada um?

Escolha o SQL quando:

Dados estruturados e estáveis

Estrutura bem definida e pouco variável

Integridade crítica

Transações financeiras, inventário, faturação

Queries complexas

Necessidade de JOINS, agregações sofisticadas

Conformidade regulamentar

RGPD, auditoria, rastreabilidade

Equipa experiente em SQL

Aproveitamento de competências existentes

📌 **Exemplos:** Sistemas bancários, ERP, CRM, gestão de stocks, faturação.

Escolha o NoSQL quando:

Escala massiva

Milhões de utilizadores, petabytes de dados

Alta disponibilidade

Sistema deve estar sempre acessível

Dados não estruturados

Estrutura varia ou é desconhecida à partida

Desenvolvimento ágil

Requisitos mudam frequentemente

Desempenho de leitura

Queries simples mas em volume extremo

📌 **Exemplos:** Redes sociais, IoT, análise de logs, catálogos de produtos, gaming.

Abordagem híbrida: Polyglot Persistence

Muitas organizações modernas não escolhem apenas uma tecnologia. Em vez disso, utilizam **diferentes bases de dados para diferentes necessidades**, uma abordagem chamada **persistência poliglota**.

Exemplo prático

Uma plataforma de e-commerce pode usar:

PostgreSQL

Para gestão de encomendas e pagamentos (consistência)

MongoDB

Para catálogo de produtos (flexibilidade)

Redis

Para cache de sessões (velocidade)

Elasticsearch

Para pesquisa de produtos (full-text search)

Esta abordagem maximiza as vantagens de cada tecnologia, mas aumenta a complexidade operacional.

Matriz comparativa com os critérios de decisão

Critério	SQL	NoSQL
Estrutura de dados	Rígida, predefinida	Flexível, adaptável
Escalabilidade	Vertical (servidor maior)	Horizontal (mais servidores)
Consistência	Forte (ACID)	Eventual (BASE)
Queries complexas	Excelente	Limitada
Velocidade (leituras simples)	Boa	Excelente
Transações	Robustas, multi-tabela	Limitadas, geralmente por documento
Curva de aprendizagem	Moderada	Varia por tipo
Maturidade	Muito elevada	Crescente
Custo de escala	Elevado	Moderado

ESTUDO DE CASO

Sistema de gestão para uma clínica médica

Contexto

⚠ Uma clínica médica pretende digitalizar os seus processos. Atualmente, utiliza papel e ficheiros Excel dispersos.

Precisa de gerir:

- **Pacientes:** Dados pessoais, historial médico
- **Consultas:** Agendamento, notas clínicas
- **Médicos:** Especialidades, horários
- **Faturação:** Pagamentos, seguros
- **Medicação:** Prescrições, alergias

A clínica tem 5 médicos, cerca de 2 000 pacientes ativos e realiza aproximadamente 150 consultas por semana.

PASSO 1

Identificar os requisitos críticos

Vamos questionar o cenário para perceber as prioridades.

Pergunta	Resposta
Que operações são mais críticas?	1. Integridade nos dados de faturação (não pode haver erros) 2. Histórico médico completo e fiável (responsabilidade legal) 3. Agendamento sem conflitos (um médico, uma consulta de cada vez)
Qual é o volume de dados esperado?	1. Volume moderado (milhares de registos, não milhões) 2. Crescimento previsível (linear com novos pacientes)
Os dados têm estrutura estável?	1. Sim. Pacientes têm sempre nome, NIF, contacto, data de nascimento 2. Consultas têm sempre data, médico, paciente, diagnóstico 3. Prescrições seguem formato regulamentado

PASSO 2

Avaliar os dados face aos modelos

Características dos dados

1 Relações claras

Um paciente tem múltiplas consultas,
cada consulta tem um médico

2 Integridade referencial

Se eliminar um médico, que acontece às
suas consultas?

3 Transações

Marcar consulta + Reservar horário +
Enviar confirmação = Operação atómica

4 Consistência

Dados médicos não podem estar "eventualmente consistentes"

5 Queries complexas

"Quantas consultas de cardiologia tivemos em dezembro com
pacientes diabéticos?"

 **Conclusão:** Estes dados encaixam perfeitamente no modelo relacional.

PASSO 3

Considerar o NoSQL, faz sentido?

Vamos testar mentalmente cada tipo de NoSQL.

Documentos (MongoDB)

- × Não há benefício em estrutura flexível (os dados são previsíveis)
- × Não precisamos de escala horizontal
- × Dificulta queries relacionais ("Consultas do Dr. Silva em janeiro")

Colunas (Cassandra)

- × Exagero para o volume de dados
- × Otimização para análise não é prioritária

Chave-Valor (Redis)

- × Demasiado simples para as relações necessárias
- ✓ Poderia servir como cache (sessões de utilizadores do sistema)

Grafos (Neo4j)

- × Não temos necessidade de queries sobre redes de relações
- × As relações são simples e hierárquicas (1-para-muitos)

PASSO 4

Decisão fundamentada

Escolha: Base de dados SQL (PostgreSQL)

Justificação:

1 Integridade crítica

ACID garante que pagamentos e dados históricos são consistentes

3 Queries complexas

Relatórios de gestão exigem JOINS entre múltiplas tabelas

5 Competências

Mais fácil encontrar programadores SQL em Portugal

2 Relações naturais

Paciente ↔ Consulta ↔ Médico são relacionamentos perfeitos para SQL

4 Conformidade legal

Auditoria e rastreabilidade de alterações (RGPD)

6 Volume adequado

Não justifica a complexidade de NoSQL

📌 **Possível componente NoSQL:** Redis para cache de sessões web e dados temporários (opcional, otimização futura).

Exemplo de modelação

Como ficaria em SQL?

Tabela Pacientes

- id (chave primária)
- nome
- nif
- data_nascimento
- contacto
- morada

Tabela Medicos

- id (chave primária)
- nome
- especialidade
- numero_ordem

Tabela Consultas

- id (chave primária)
- paciente_id (chave estrangeira → Pacientes)
- medico_id (chave estrangeira → Medicos)
- data_hora
- diagnostico
- notas

Relações garantidas:

- Não posso marcar consulta sem paciente válido
- Não posso apagar médico se tiver consultas associadas
- Uma transação garante Agendar + Bloquear horário + Notificar

Exemplo de modelação (cont.)

Como ficaria em NoSQL (MongoDB)? - Só para comparação

```
{
  "_id": "consulta_001",
  "data": "2026-01-15T10:00:00",
  "paciente": {
    "nome": "João Silva",
    "nif": "123456789",
    "contacto": "912345678"
  },
  "medico": {
    "nome": "Dra. Ana Costa",
    "especialidade": "Cardiologia"
  },
  "diagnostico": "Hipertensão controlada",
  "prescricoes": [...]
}
```

Problemas evidentes

- Se o Dr.^a Ana mudar de nome, teria de atualizar TODAS as consultas
- Não há garantia de que o NIF é único
- Difícil saber "Quantas consultas tem o paciente X?" sem varrer documentos

Principais aprendizagens

1 O volume de dados não é o único fator

A clínica tem poucos dados, mas isso não torna NoSQL inadequado, são as **relações e consistência** que decidem

2 Estrutura estável favorece SQL

Dados médicos seguem regulamentação, logo têm formato previsível

3 ACID não é negociável em certos contextos

Faturação e saúde exigem uma consistência forte

4 NoSQL não é sempre "melhor" ou "moderno"

É uma ferramenta para problemas específicos (escala, flexibilidade)

5 Polyglot Persistence é realista

Podemos adicionar o Redis para cache, sem abandonar o PostgreSQL

Glossário

ACID: Propriedades de transações em bases de dados relacionais (Atomicidade, Consistência, Isolamento, Durabilidade)

BASE: Modelo de consistência de sistemas NoSQL (Basically Available, Soft state, Eventually consistent)

Chave estrangeira: Campo que referencia a chave primária de outra tabela, estabelecendo relações

Chave primária: Identificador único de cada registo numa tabela

Escalabilidade horizontal: Adicionar mais servidores para aumentar capacidade

Escalabilidade vertical: Aumentar recursos (CPU, RAM) do servidor existente

JSON: Formato de texto para representar dados estruturados (JavaScript Object Notation)

Normalização: Processo de organizar dados para eliminar redundância

Polyglot Persistence: Utilização de diferentes tipos de bases de dados na mesma aplicação

Schema: Estrutura que define como os dados são organizados (tabelas, colunas, tipos)

Transação: Conjunto de operações que são executadas como uma unidade (tudo ou nada)

Questões frequentes

“

Se NoSQL é mais moderno, não deveria ser sempre a escolha?

Não. "Moderno" não significa "melhor para tudo". NoSQL surgiu para resolver problemas específicos (escala massiva, dados não estruturados) que a maioria das empresas não tem. SQL continua a ser a escolha mais adequada para a grande maioria dos casos empresariais.

”

“

Posso migrar de SQL para NoSQL mais tarde?

Tecnicamente sim, mas é um processo complexo e arriscado. A modelação de dados é completamente diferente. É fundamental escolher bem à partida, considerando não só o presente mas o crescimento esperado.

”

“

MongoDB é sempre mais rápido que PostgreSQL?

Não. Para queries simples de leitura, pode ser, mas para queries complexas com múltiplas relações, SQL geralmente supera NoSQL. O desempenho depende do caso de utilização específico.

”

“

Posso usar SQL e NoSQL ao mesmo tempo?

Sim, chama-se polyglot persistence. Por exemplo, PostgreSQL para transações e MongoDB para logs. Mas aumenta significativamente a complexidade operacional (dois sistemas para manter, sincronizar, fazer backup).

”

“

O NoSQL não tem queries?

Tem, mas são diferentes. Cada tipo de NoSQL tem a sua linguagem. Por exemplo, MongoDB usa uma sintaxe semelhante a JavaScript. Contudo, queries com múltiplas relações (JOINS) são muito limitadas comparadas com SQL.

”

“

As bases de dados NoSQL são sempre distribuídas?

Não obrigatoriamente, mas é uma das suas grandes vantagens. Estão desenhadas para facilitar distribuição por múltiplos servidores, algo que SQL faz com mais dificuldade.

”

“

O Excel não é uma base de dados?

Tecnicamente pode guardar dados tabulares, mas não é um DBMS (sistema de gestão de bases de dados). Não garante consistência, não suporta acessos simultâneos adequadamente, não tem controlos de integridade. Para utilização profissional, é inadequado como base de dados principal.

”